

Detecção de Falhas em Drones com Base em Sinais de Vibração Utilizando TinyML

André Alves de Freitas¹, Jeandro de Mesquita Bezerra¹

¹Universidade Federal do Ceará (UFC) - Av. José de Freitas Queiroz, 5003, 63902-580
Cedro – Quixadá – Ceará - Brasil

freitasalvesandrel6@alu.ufc.br, jeandro@ufc.br

1. Introdução

Nos últimos anos, os drones têm sido cada vez mais usados em tarefas como inspeção, mapeamento e monitoramento de ambientes. Com isso, garantir que esses equipamentos funcionem de forma confiável ficou mais importante do que antes. Os motores de propulsão são peças centrais pra isso: se algo der errado com eles, o drone pode perder estabilidade ou até cair. Por isso, detectar problemas nesses componentes o mais cedo possível é algo que tem chamado bastante atenção na área [Adaika et al. 2025, Puchalski and Giernacki 2022].

Uma forma bastante usada pra monitorar esse tipo de sistema é através de sinais de vibração. A ideia é que quando algo muda no funcionamento mecânico do motor, isso aparece como uma mudança no padrão de vibração. Isso é interessante porque não precisa mexer no motor pra fazer essa análise. Alguns trabalhos recentes já mostraram que dá pra usar acelerômetros junto com técnicas de aprendizado de máquina pra identificar problemas em hélices e motores de drone com esse tipo de dado [Tiboni et al. 2022, Al-Haddad et al. 2024, Ozkat 2024].

Outra coisa que tem avançado bastante é o TinyML, que é basicamente a execução de modelos de machine learning em microcontroladores e dispositivos com poucos recursos. A vantagem disso é que o processamento fica próximo de onde os dados são coletados, sem precisar ficar enviando tudo pra nuvem ou pra um servidor externo. Claro que isso também traz desafios, porque esses dispositivos têm limitações sérias de memória, energia e processamento, então é preciso escolher bem os sensores, os atributos e os modelos que serão usados [Lin et al. 2023, David et al. 2021].

Neste trabalho, a proposta é justamente desenvolver um sistema de detecção de falhas em drone a partir de sinais de vibração coletados por um embarcado. O sensor usado pra isso é o MPU-6050, e o objetivo é conseguir diferenciar três condições de operação do motor: normal, anômalo nível 1 e anômalo nível 2. Com isso, pretende-se criar uma base inicial pra investigar o uso de TinyML no monitoramento embarcado de drones.

2. Trabalhos Relacionados

Nesta seção, serão apresentados e comparados os principais trabalhos relacionados ao tema proposto neste estudo.

2.1. *Vibration data-driven anomaly detection in UAVs: A deep learning approach*

[Ozkat 2024] propõem uma abordagem baseada em aprendizado profundo para a detecção precoce de anomalias em VANTs, com foco específico em falhas estruturais nos propulsores. O problema central abordado é a dificuldade de identificar, em tempo real, degradações mecânicas nos propulsores antes que evoluam para falhas catastróficas durante o voo. Dado que estudos apontam que aproximadamente 67% dos acidentes com VANTs decorrem de falhas no sistema mecânico, e que mais da metade dessas falhas ocorre no sistema de propulsão, a necessidade de métodos eficazes de monitoramento contínuo é evidente. Diante desse cenário, os autores propõem uma rede denominada *Wavelet Scattering Long Short-Term Memory Autoencoder* (WS-LSTM-AE), que combina a transformada de espalhamento por *wavelets* com um *autoencoder* baseado em LSTM, visando capturar simultaneamente informações nos domínios do tempo e da frequência a partir de sinais de vibração.

Para o desenvolvimento do trabalho, foram realizados experimentos controlados utilizando um VANT multirrotor DJI M600, equipado com seis rotores. A fim de simular uma falha real de propulsor, uma das pás foi modificada com uma ranhura de 1,5 mm de profundidade posicionada a 9 cm da ponta, cobrindo aproximadamente 75% da espessura da pá. Os sinais de vibração foram capturados pela *Inertial Measurement Unit* (IMU) Xsens MTi-G-700, operando a uma taxa de amostragem de 800 Hz, com os dados coletados nos três eixos cartesianos (x, y e z) ao longo de dez posições de acelerador predefinidas. O experimento foi replicado três vezes para garantir a confiabilidade dos resultados. A rede proposta opera de maneira não supervisionada: treinada exclusivamente com dados normais, ela aprende a reconstruir padrões de operação saudável e detecta anomalias quando o erro de reconstrução, medido pelo Erro Absoluto Médio, ultrapassa um limiar definido com base no intervalo interquartil dos dados de validação.

Os resultados obtidos demonstraram que a rede foi capaz de emitir alertas antecipados de falha em todas as réplicas e em todos os eixos analisados, com o eixo y apresentando os primeiros avisos e o eixo z os mais tardios, comportamento atribuído ao desequilíbrio de massa dos propulsores no plano XY. O tempo de antecedência dos alertas variou entre 90 e 100 segundos antes da ocorrência da falha nas três réplicas, o que os autores consideram uma janela suficiente para a execução de manobras de segurança, como o pouso de emergência. Entre as principais contribuições do trabalho, destaca-se a eliminação da necessidade de extração manual de características do sinal, bem como a viabilidade de operação em tempo real sem dependência de dados rotulados. Como limitação reconhecida pelos próprios autores, o estudo foi conduzido em ambiente laboratorial controlado com um único modelo de VANT, o que pode restringir a generalização dos resultados para cenários operacionais mais diversos.

2.2. *Cost effective detection of uneven mounting fault in rotary wing drone motors with a CNN based method*

[Ceylan et al. 2024] abordam um problema específico e pouco explorado na literatura: a detecção de falhas de montagem irregular em motores *Brushless Direct Current* (BLDC) utilizados em drones de asa rotativa. Esse tipo de falha ocorre quando o motor não é fixado corretamente na estrutura do drone durante a fase de montagem, gerando vibrações anômalas que podem comprometer a estabilidade do voo e ocasionar acidentes. O trabalho se insere no contexto de manutenção preditiva e controle de qualidade na montagem

de VANTs, e a proposta central dos autores é desenvolver um método de detecção baseado em aprendizado profundo que seja, ao mesmo tempo, preciso e economicamente viável para aplicação em sistemas embarcados.

A metodologia do trabalho envolveu a construção de um bancada de testes com um motor BLDC modelo A2212/10T 1400 KV operando em três rotações distintas, sendo 2000, 4000 e 6000 rpm, com e sem a condição de montagem irregular. Os sinais de vibração foram coletados por um acelerômetro MEMS de baixo custo, o ADXL-345, nos eixos X e Y, sendo o eixo Z descartado por apresentar menor variância informativa. Para o treinamento do modelo, os dados coletados na bancada foram combinados com o conjunto de referência da *Case Western Reserve University (CWRU)*, e o volume de amostras foi ampliado por meio da técnica de janela deslizante. A rede neural utilizada foi a WDD-CNN, uma arquitetura que integra blocos convolucionais unidimensionais e bidimensionais em paralelo, combinando as vantagens de ambas as abordagens. Após o treinamento em ambiente computacional convencional, os pesos da rede foram transferidos para um microcontrolador *Raspberry Pi 5*, onde o processo de classificação foi executado em tempo real.

Os resultados obtidos nos dois experimentos realizados, um com dados a 4000 rpm e outro a 6000 rpm, demonstraram acurácia de 100%, com precisão, *recall* e *F1-score* iguais a 1 em ambos os casos. Uma contribuição relevante do trabalho está no fato de ser, segundo os próprios autores, o primeiro estudo a tratar especificamente da falha de montagem irregular em motores BLDC de drones utilizando aprendizado profundo, preenchendo uma lacuna identificada na literatura. Além disso, a execução da classificação diretamente no microcontrolador, sem a necessidade de processamento externo, reduz o custo operacional do sistema, uma vez que os drones já possuem esse componente embarcado. Como limitação reconhecida pelos autores, a utilização de dados sintéticos gerados por aumento no treinamento pode não refletir plenamente as condições reais de operação, e a abordagem ainda não distingue a falha de montagem de outras fontes de vibração, como condições climáticas ou defeitos mecânicos concomitantes.

2.3. IoT device for detecting abnormal vibrations in motors using TinyML

[Arciniegas et al. 2025] apresentam uma abordagem voltada à detecção de falhas em rolamentos de motores industriais por meio de um dispositivo IoT baseado em *TinyML*, com ênfase na viabilidade de execução em microcontroladores de baixo consumo energético. O problema central abordado é a necessidade de realizar manutenção preditiva em tempo real sem depender de infraestrutura de nuvem robusta, o que limita a aplicabilidade em ambientes industriais com restrições de conectividade e latência. O trabalho se insere no contexto da computação de borda aplicada à Internet das Coisas (*IoT*), e a proposta principal consiste em desenvolver um sistema capaz de classificar o estado operacional do motor e detectar anomalias vibratórias diretamente no dispositivo embarcado, combinando análise espectral via FFT com uma rede neural profunda compacta e um algoritmo de agrupamento *K-Means*.

Para a coleta de dados, foi utilizado o acelerômetro MPU6050, operando a uma frequência de amostragem de 100 Hz nos eixos X, Y e Z, instalado em uma bancada com um motor de ventilador como ambiente de teste. O sinal bruto foi segmentado em janelas temporais com 50% de sobreposição, e a FFT foi aplicada com janelas de 16 amostras

para extração de características no domínio da frequência. A plataforma *Edge Impulse* foi utilizada para o treinamento e otimização do modelo, que consiste em uma rede neural com duas camadas ocultas de 20 e 10 neurônios, respectivamente. Para viabilizar a execução no microcontrolador XIAO ESP32S3, foram aplicadas técnicas de quantização dos pesos de 32 para 8 bits e poda por fatoração matricial. A detecção de anomalias foi realizada de forma paralela à classificação, utilizando o algoritmo *K-Means* com 32 agrupamentos. Os resultados de inferência foram transmitidos via protocolo MQTT para o *Node-RED*, com geração de alertas por meio de um *bot* no *Telegram* e armazenamento dos dados no *InfluxDB*.

O modelo *TinyML* desenvolvido alcançou acurácia de 96,5%, precisão de 96,19%, taxa de *recall* de 97,11% e *F1 score* de 96,64% em condições laboratoriais, superando os modelos de referência *Random Forest* (93,5%) e SVM com *kernel* RBF (89,8%). O tempo médio de inferência foi de 25 ms, e a latência total do sistema, desde a coleta até a geração do alerta, foi de aproximadamente 300 ms. Em testes de operação real ao longo de dois meses, o sistema manteve acurácia de detecção de 98%. Como contribuição relevante, o trabalho demonstra a viabilidade de executar modelos de aprendizado de máquina sofisticados em dispositivos de baixo custo e consumo reduzido, dispensando processamento externo. Os próprios autores reconhecem como limitações a sensibilidade a ruídos ambientais provenientes de maquinário adjacente, que gerou alguns falsos positivos, além da restrição de escalabilidade para o monitoramento simultâneo de múltiplos motores.

2.4. Análise Comparativa

[Ozkat 2024] utiliza sinais de vibração capturados pela IMU para detecção de anomalias nas pás da hélice em VANTs, o que se alinha diretamente com a abordagem proposta neste trabalho. Contudo, o método é executado em ambiente computacional convencional, sem inferência local, o que introduz latência considerável no processo de detecção e tem o custo mais elevado devido ao uso da IMU MTi-G-700. Neste trabalho, por sua vez, tem o custo reduzido e a inferência é realizada diretamente no dispositivo embarcado por meio de *TinyML*, eliminando a dependência de processamento externo e tornando a solução mais adequada para aplicações em VANTs.

Em [Ceylan et al. 2024], a detecção de falhas de montagem em motores BLDC de drones é realizada com sinais de vibração e uma rede WDD-CNN transferida para uma Raspberry Pi 5, alcançando 100% de acurácia. Embora a classificação ocorra em hardware embarcado, a Raspberry Pi 5 possui capacidade computacional muito superior à de microcontroladores típicos de baixo custo, o que eleva o custo e o consumo energético da solução. Neste trabalho, o foco recai sobre dispositivos mais restritos, com uso explícito de *TinyML* para viabilizar a execução em microcontroladores acessíveis e uso das hélices do drone como objeto de estudo.

[Arciniegas et al. 2025] propõem um dispositivo IoT com *TinyML* para detecção de anomalias em motores de ventilador, utilizando o acelerômetro MPU6050 e o microcontrolador ESP32S3, com inferência local e latência de 300 ms. Esse trabalho compartilha os principais aspectos técnicos com a proposta aqui apresentada, como o uso de *TinyML*, sensor de vibração de baixo custo e inferência embarcada. A distinção central está no domínio de aplicação, enquanto o trabalho relacionado foca em motores em bancada, este direciona a solução para a detecção de falhas em hélices de drones em condições reais de operação.

A Tabela 1 expõem um resumo das principais características dos trabalhos relacionados descritos anteriormente em comparação com o trabalho proposto.

Tabela 1. Tabela comparativa

Trabalho	Foco em falhas nas hélices de drone	Uso de <i>TinyML</i>	Inferência local	Utiliza sinais de vibração	Baixo custo
[Ozkat 2024]	Sim	Não, utiliza WS-LSTM-AE	Não	Sim	Não, devido ao uso da IMU MTi-G-700
[Ceylan et al. 2024]	Não, foco em montagem irregular do motor	Não, usa CNN convencional	Sim, classificação com a <i>Raspberry Pi 5</i>	Sim	Parcial
[Arciniegas et al. 2025]	Não, foco em motor de ventilador	Sim, <i>TinyML</i> com <i>Edge Impulse</i>	Sim, inferência com a ESP32S3	Sim	Sim
Trabalho proposto	Sim	Sim, usa <i>TinyML</i> com a rede CNN-1D otimizada	Sim, inferência com a <i>Raspberry pi pico w</i>	Sim	Sim

3. Problema

O problema que esse trabalho trata é a detecção de falhas em motores de drone usando sinais de vibração. Em vez de depender de inspeções manuais ou monitoramento externo, a ideia é identificar diferentes condições de operação a partir de dados do próprio sistema. Faz sentido pensar assim porque mudanças no comportamento mecânico do motor tendem a aparecer nos sinais de vibração de forma observável, o que é compatível com abordagens de diagnóstico que já foram exploradas na literatura [Adaika et al. 2025, Al-Haddad et al. 2024, Tiboni et al. 2022].

Pra capturar esses sinais, o trabalho usa o sensor MPU-6050. A hipótese é que os três estados iniciais de operação definidos aqui (*normal*, *anômalo nível 1* e *anômalo nível 2*) produzem padrões vibracionais distintos o suficiente pra serem separados por um classificador. Com isso, o problema vira uma tarefa de classificação supervisionada, onde cada amostra de vibração é associada a uma dessas classes, representando a condição atual do motor.

O uso de *TinyML* faz sentido nesse cenário principalmente pelas restrições que existem em sistemas embarcados em drones. Rodar o modelo direto no microcontrolador reduz a latência, diminui a dependência de conexão contínua e evita ficar transmitindo dados brutos o tempo todo. Além disso, o *TinyML* é mais compatível com as limitações de memória e energia que são comuns nesses dispositivos [Lin et al. 2023, David et al. 2021]. Então, o problema deste trabalho está dentro do domínio de VANT (Veículo Aéreo Não Tripulado), com foco em monitoramento de condição e detecção de anomalias.

4. Requisitos do sistema/protótipo

O sistema tem como objetivo coletar sinais de vibração de motores de drone por um embarcado e disponibilizar esses dados pra treinar modelos em um ambiente externo. Depois disso, a ideia é implantar o modelo treinado de volta no embarcado, pra fazer inferência com *TinyML*. O protótipo é composto por um microcontrolador, um sensor MPU-6050 pra aquisição dos sinais e um motor de drone como elemento monitorado.

Como requisitos funcionais, o sistema precisa: (i) adquirir sinais de vibração do motor pelo sensor MPU-6050; (ii) registrar amostras em diferentes condições de

operação; (iii) armazenar ou transmitir os dados coletados pra posterior organização; (iv) permitir associar os sinais às condições experimentais de cada coleta; e (v) fornecer uma base de dados compatível com as etapas de pré-processamento, extração de características, treinamento e futura execução de modelos de TinyML.

As três condições de operação consideradas são: *normal*, que é o funcionamento esperado do motor e *anômalo*, quando há alguma alteração perceptível no padrão de vibração (essa condição foi dividida em 2 níveis de desbalanceamento das hélices). Essas classes foram escolhidas pra cobrir tanto estados de referência quanto situações mais críticas.

Em relação aos requisitos não funcionais, o sistema precisa coletar os sinais com resolução temporal suficiente pra representar bem o comportamento do motor, sem extrapolar a capacidade do embarcado. Os modelos também precisam ser leves, já que memória e processamento são limitados. O consumo de energia é um fator importante, especialmente em drones onde a bateria é um recurso crítico. Por fim, a fixação do sensor precisa ser estável, porque qualquer variação no posicionamento pode afetar a qualidade dos dados.

A plataforma escolhida é um microcontrolador com o sensor MPU-6050 e um motor de drone. Essa configuração foi adotada pelo baixo custo, facilidade de montar e pela possibilidade de reproduzir diferentes condições de operação de forma controlada. Também é uma arquitetura que facilita a transição da etapa de coleta pra etapa de inferência com TinyML lá na frente.

5. Dataset

O conjunto de dados utilizado neste trabalho foi construído a partir de sinais de vibração coletados pelo próprio protótipo desenvolvido, diferente do Entregável 2, em que o treinamento do modelo havia sido feito sobre o dataset público de referência [Al-Haddad 2025]. Naquela etapa, o dataset de referência serviu principalmente como inspiração para a definição das condições anômalas, já que ele reúne sinais de vibração coletados por um acelerômetro triaxial acoplado a um drone real em operação, simulando desbalanceamento nas hélices por meio da adição de fita adesiva.

Para este entregável, optou-se por treinar o modelo exclusivamente com o dataset próprio, coletado com o sensor MPU-6050 acoplado a um motor de drone montado em bancada. Essa escolha foi feita porque o objetivo principal desta etapa é validar o pipeline completo de TinyML, da coleta de dados até a inferência embarcada, sendo importante que o modelo treinado corresponda exatamente ao sensor, à taxa de amostragem e às condições físicas do protótipo construído.

5.1. Coleta dos Dados

A coleta foi realizada cobrindo as mesmas três condições de operação definidas anteriormente:

- **Normal:** motor funcionando normalmente, com hélice balanceada.
- **Anômalo nível 1:** hélice com pequeno desbalanceamento, simulado com fita adesiva em menor quantidade.
- **Anômalo nível 2:** hélice com desbalanceamento maior, simulado com mais massa de fita adesiva.

Em cada condição, o motor foi simplesmente ligado e mantido em funcionamento constante durante a coleta, sem um controle preciso de rotação. Isso significa que a rotação do motor não foi medida nem fixada em um valor exato de RPM, o que constitui uma limitação semelhante à já apontada no Entregável 2, em que a especificação exata das condições de teste em termos de rotação também havia ficado pendente.

Diferente do dataset de referência, em que a taxa de amostragem chegava a cerca de 1023 Hz, a coleta própria foi limitada a aproximadamente 100 Hz. Essa limitação não é do sensor MPU-6050 em si, mas sim da comunicação serial USB utilizada para transmitir os dados do microcontrolador Raspberry Pi Pico W para o computador durante a fase de aquisição. Testes preliminares mostraram que a interface USB CDC do Pico W ignora a taxa de baud configurada, impondo um teto prático de transmissão em torno de 315 a 400 Hz, e que valores mais conservadores, como 100 Hz, garantem maior confiabilidade na coleta sem perda de amostras.

A coleta gerou um total de 45.080 amostras brutas, distribuídas de forma aproximadamente equilibrada entre as três classes, conforme a Tabela 3.

Tabela 2. Distribuição de amostras brutas por classe

Classe	Amostras	Proporção
Normal	15.019	33,3%
Anômalo N1	15.030	33,3%
Anômalo N2	15.031	33,4%
Total	45.080	100%

5.2. Estrutura e Desafios

O volume de amostras coletadas neste entregável é consideravelmente menor do que o do dataset de referência, refletindo diretamente a diferença entre a taxa de amostragem de aproximadamente 1023 Hz daquele dataset e os 100 Hz alcançados pela comunicação serial do protótipo. Essa diferença teve impacto direto sobre o tamanho de janela escolhido, conforme detalhado na Seção 9.

Os principais desafios enfrentados durante a coleta foram semelhantes aos previstos no Entregável 2: manter a fixação estável do sensor entre as sessões de coleta de cada classe, reproduzir a mesma quantidade e posição da fita adesiva nas condições anômalas, e lidar com a restrição de taxa de amostragem imposta pela transmissão serial. Esse último ponto se mostrou mais limitante do que o esperado, já que reduziu significativamente a resolução temporal disponível em comparação ao dataset público utilizado anteriormente. Além disso, a ausência de controle sobre a rotação do motor limita a capacidade de relacionar diretamente a intensidade do desbalanceamento detectado com uma condição de operação específica em termos de RPM.

6. Definição do Modelo

Dado que o dataset é composto por séries temporais de sinais de vibração segmentadas em janelas, o modelo escolhido para esse tipo de problema continua sendo a CNN 1D [RAJAKANNU et al. 2024], mantendo a mesma arquitetura definida no Entregável 2. Essa arquitetura aprende padrões locais diretamente na série temporal, sem precisar de

extração manual de features, o que é interessante para dados de vibração porque características relevantes de frequência e amplitude aparecem naturalmente nos filtros convolucionais durante o treinamento.

A principal diferença em relação ao entregável anterior está na dimensão de entrada da rede, que passou de janelas de 256 amostras para janelas de 26 amostras, conforme detalhado na Seção 9. Essa mudança decorre da taxa de amostragem efetiva alcançada pelo protótipo embarcado, bem mais baixa do que a do dataset de referência. Para acomodar essa entrada menor sem alterar a estrutura geral da rede, foi adotado preenchimento do tipo *same* nas camadas convolucionais, permitindo que o terceiro bloco convolucional, com kernel de tamanho 3, continuasse aplicável mesmo após a dimensão temporal ser reduzida para 1 pelas camadas de MaxPooling anteriores.

Apesar dessa adaptação, o modelo manteve exatamente os mesmos 6.467 parâmetros treináveis e o mesmo tamanho de aproximadamente 25 KB em ponto flutuante de 32 bits do entregável anterior, o que reforça a robustez da arquitetura escolhida frente a mudanças na resolução temporal da entrada.

7. Justificativa Técnica

As escolhas feitas nesse projeto foram pensadas pra ser compatíveis com o cenário de detecção de falhas em drones e com as limitações típicas de sistemas embarcados, além de serem viáveis pra implementar em contexto acadêmico. O problema de detecção de falhas em drones foi escolhido porque esse equipamento tem vários componentes que são fundamentais pra estabilidade e segurança do sistema. Identificar comportamentos anormais cedo pode evitar falhas em voo e aumentar a confiabilidade geral do equipamento, o que é tecnicamente relevante pra aplicações de monitoramento de VANTs [Adaika et al. 2025, Al-Haddad et al. 2024].

Usar sinais de vibração faz sentido porque, em sistemas rotativos, mudanças nas condições de operação costumam aparecer no padrão vibracional. Além disso, é uma abordagem não intrusiva, ou seja, dá pra observar o comportamento do motor sem precisar abrir ou modificar nada nele. Isso é coerente com o que a literatura de monitoramento de condição apresenta [Tiboni et al. 2022].

O MPU-6050 foi escolhido por ser um sensor inercial simples, barato e amplamente disponível. Ele fornece leituras de aceleração e velocidade angular, o que é suficiente pra capturar as vibrações do motor. A integração com microcontroladores é fácil, o que facilita muito a montagem do protótipo.

A opção de construir um dataset próprio, em vez de usar uma base pública, foi feita porque isso garante que os dados sejam diretamente relacionados ao sistema que está sendo monitorado. Assim, tem-se mais controle sobre o processo de coleta e sobre o que cada classe representa. É verdade que isso traz desafios, como menor volume de dados e dificuldade em definir bem as condições anômalas, mas a coerência entre aquisição e proposta de implementação embarcada justifica essa escolha.

Por fim, o TinyML é a abordagem adotada porque o objetivo futuro é rodar o modelo diretamente no microcontrolador. Em drones, poder fazer inferência localmente é importante pra reduzir latência e não depender de conexão constante. As restrições de memória, energia e processamento desses dispositivos tornam necessário usar modelos

leves, o que é exatamente o que o TinyML propõe [Lin et al. 2023, David et al. 2021].

8. Implementação do Modelo

A implementação foi feita em Python utilizando o framework TensorFlow com a API Keras [Abadi et al. 2015], no mesmo ambiente de desenvolvimento utilizado no Entregável 2, o Google Colab. A escolha do Keras se deu pela facilidade de construir e modificar arquiteturas de redes neurais de forma rápida, além de ter boa integração com ferramentas de exportação para TinyML, como o TensorFlow Lite.

A arquitetura escolhida foi a mesma CNN 1D compacta utilizada no entregável anterior, inspirada no exemplo de classificação de séries temporais disponível na documentação oficial do Keras [Fawaz 2020]. O modelo recebe como entrada janelas de sinal com dimensão (26, 3), onde 26 representa o número de amostras temporais coletadas pelo protótipo a aproximadamente 100 Hz, e 3 corresponde aos eixos de aceleração A_x , A_y e A_z . A saída continua sendo um vetor de probabilidades para as três classes: *Normal*, *Anômalo N1* e *Anômalo N2*.

A rede é composta pelos mesmos três blocos convolucionais seguidos de um classificador mínimo. Cada bloco segue o padrão Conv1D $\rightarrow$ BatchNormalization $\rightarrow$ ReLU, com MaxPooling de tamanho 4 aplicado após os dois primeiros blocos. A única alteração estrutural em relação ao Entregável 2 foi o uso de preenchimento *same* nas convoluções, necessário para que o terceiro bloco convolucional, com kernel de tamanho 3, pudesse operar mesmo após a dimensão temporal ter sido reduzida a apenas 1 pelas camadas de MaxPooling anteriores. Após o terceiro bloco convolucional, uma camada de GlobalAveragePooling resume o sinal em um vetor de 32 features, que passa por Dropout antes da camada densa de saída com ativação Softmax.

O modelo resultante manteve os 6.467 parâmetros treináveis e o tamanho de aproximadamente 25 KB em ponto flutuante de 32 bits, idêntico ao obtido no Entregável 2.

Os parâmetros de treinamento utilizados foram os mesmos do entregável anterior: otimizador Adam com taxa de aprendizado inicial de 1×10^{-3} , função de perda *categorical crossentropy*, 20 épocas máximas e *batch size* de 64, com os mesmos três *callbacks* configurados (*EarlyStopping*, *ReduceLROnPlateau* e *ModelCheckpoint*).

A estratégia de validação também seguiu a mesma combinação de três abordagens complementares já descrita no Entregável 2: monitoramento da perda e acurácia ao longo das épocas no conjunto de validação, avaliação quantitativa no conjunto de teste por meio do *model.evaluate*, e análise da matriz de confusão para verificar a distribuição dos erros entre as classes.

9. Pré-processamento dos Dados

Assim como no Entregável 2, antes de treinar o modelo os dados passaram por etapas de preparação para garantir que a entrada da rede estivesse no formato correto e sem inconsistências. Neste entregável, essas etapas foram aplicadas sobre o dataset próprio descrito na Seção ?? anterior, coletado pelo protótipo embarcado.

9.1. Carregamento e Limpeza

Os arquivos com os sinais coletados pelo MPU-6050 foram carregados com a biblioteca Pandas, organizando as colunas em *tempo*, *acc_x*, *acc_y* e *acc_z*, e associando cada amos-

tra a um rótulo de classe (*Normal*, *Anômalo N1* ou *Anômalo N2*) conforme a condição de coleta. Ao final do carregamento, o conjunto reuniu 45.080 amostras brutas, distribuídas entre as três classes conforme apresentado na Tabela 3.

Tabela 3. Distribuição de amostras brutas por classe

Classe	Amostras	Proporção
Normal	15.019	33,3%
Anômalo N1	15.030	33,3%
Anômalo N2	15.031	33,4%
Total	45.080	100%

9.2. Segmentação em Janelas

Diferente do Entregável 2, em que a janela foi definida com 256 amostras a partir de uma taxa de amostragem de aproximadamente 1023 Hz, neste entregável a janela foi reduzida para 26 amostras, em função da taxa de amostragem efetiva de aproximadamente 100 Hz alcançada pelo protótipo. Essa configuração mantém uma duração de janela próxima à utilizada anteriormente, em torno de 260 ms, o que preserva a comparabilidade entre as duas abordagens em termos de resolução temporal do sinal capturado.

O passo entre janelas foi mantido igual ao tamanho da janela, ou seja, sem sobreposição, garantindo que cada amostra apareça em apenas uma janela e evitando vazamento de informação entre janelas adjacentes. Esse processo gerou um total de 1.733 janelas, com distribuição equilibrada entre as classes, conforme mostra a Tabela 4.

Tabela 4. Janelas geradas por classe após segmentação

Classe	Janelas
Normal	577
Anômalo N1	578
Anômalo N2	578
Total	1.733

Cada janela resultante tem dimensão (26, 3), onde 26 é o número de amostras temporais e 3 corresponde aos eixos de aceleração A_x , A_y e A_z , que são tratados como canais de entrada da rede convolucional.

9.3. Normalização

A normalização dos dados seguiu o mesmo método utilizado no Entregável 2, aplicando-se a técnica de *Z-score* por meio do *StandardScaler* da biblioteca Scikit-learn, conforme a Equação 1.

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

onde x é o valor original, μ é a média e σ é o desvio padrão do eixo correspondente. O *scaler* foi ajustado de forma global sobre o conjunto completo de janelas geradas

(*X_{all}*), antes da divisão entre treino, validação e teste, e os parâmetros de média e desvio padrão resultantes foram aplicados a todas as janelas do dataset.

É importante destacar que esse procedimento constitui um vazamento de dados entre os conjuntos de treino, validação e teste, já que os parâmetros do *scaler* foram calculados levando em conta também as janelas que mais tarde compuseram o conjunto de teste. Esse problema é distinto do vazamento já identificado e corrigido no Entregável 2, que dizia respeito à divisão aleatória das janelas em vez da divisão temporal. Aqui, mesmo com a divisão temporal correta entre treino, validação e teste, o ajuste do *scaler* sobre o conjunto completo introduz uma forma mais sutil de vazamento, já que estatísticas globais de média e desvio padrão calculadas sobre todo o dataset são utilizadas para normalizar inclusive as janelas de teste. Essa limitação é discutida com mais detalhe na Seção 12, junto à análise crítica dos resultados obtidos.

Os parâmetros de média e desvio padrão resultantes do *StandardScaler* foram extraídos do notebook e embarcados diretamente no firmware do microcontrolador, como constantes fixas por eixo, conforme detalhado na Seção 13. Essa abordagem permite que a etapa de normalização seja replicada no dispositivo embarcado sem a necessidade de recalcular estatísticas em tempo real, mas também significa que o firmware depende diretamente dos valores obtidos durante o treinamento, não se adaptando a eventuais mudanças nas condições de operação do sensor após o embarque do modelo.

9.4. Divisão dos Dados

A divisão entre treino, validação e teste manteve a mesma estratégia adotada no Entregável 2, respeitando a ordem temporal das janelas para cada classe: os primeiros 70% das janelas foram destinados ao treino, os 15% seguintes à validação e os últimos 15% ao teste, evitando que janelas temporalmente próximas aparecessem em conjuntos diferentes. A distribuição final ficou conforme a Tabela 5.

Tabela 5. Distribuição dos dados após divisão temporal

Conjunto	Janelas	Por classe	Proporção
Treino	1.217	405 / 406 / 406	70%
Validação	258	86 / 86 / 86	15%
Teste	258	86 / 86 / 86	15%
Total	1.733	577 / 578 / 578	100%

O modelo recebe como entrada janelas com dimensão (26, 3), confirmando a adequação da arquitetura à nova resolução temporal dos dados coletados pelo protótipo embarcado.

10. Treinamento e Validação do Modelo

O problema tratado neste trabalho é de classificação supervisionada, em que cada janela de sinal de vibração é associada a uma das três classes definidas: *Normal*, *Anômalo N1* e *Anômalo N2*. O modelo utilizado foi a CNN 1D compacta descrita na Seção 8, treinada com os dados próprios preparados conforme a Seção 9.

A função de ativação utilizada nas camadas convolucionais foi a ReLU (*Rectified Linear Unit*), e a camada de saída utilizou a função Softmax, responsável por transformar

os valores brutos da rede em probabilidades para cada uma das três classes. A função de perda adotada foi a *categorical crossentropy*, adequada para problemas de classificação multiclasse com rótulos em formato *one-hot*, e o otimizador utilizado foi o Adam, com taxa de aprendizado inicial de 1×10^{-3} .

O treinamento foi configurado para até 20 épocas com *batch size* de 64. Três *callbacks* foram utilizados: *EarlyStopping*, com limite de 3 épocas sem melhora na perda de validação; *ReduceLROnPlateau*, que reduz a taxa de aprendizado pela metade após 7 épocas sem melhora na validação; e *ModelCheckpoint*, para salvar os pesos da melhor época com base na acurácia de validação.

O treinamento apresentou uma convergência relativamente lenta na primeira época, com a acurácia de treino atingindo aproximadamente 75,8% e a acurácia de validação ficando em apenas 43,0%, com perda de validação de 0,93. A partir da terceira época, o modelo passou a convergir de forma consistente, atingindo acurácia de validação de 100% e perda de validação caindo para 0,52. Essa tendência se manteve estável até o final do treinamento: na vigésima época, a acurácia de treino e de validação permaneceram em 100%, com perda de treino de 0,0131 e perda de validação de 0,0053. O *EarlyStopping* não chegou a interromper o treinamento antes do limite de 20 épocas configurado, e o *ModelCheckpoint* restaurou os pesos correspondentes à última época, indicada como a melhor com base na acurácia de validação. As curvas de perda e acurácia ao longo do treinamento são apresentadas na Figura 1.

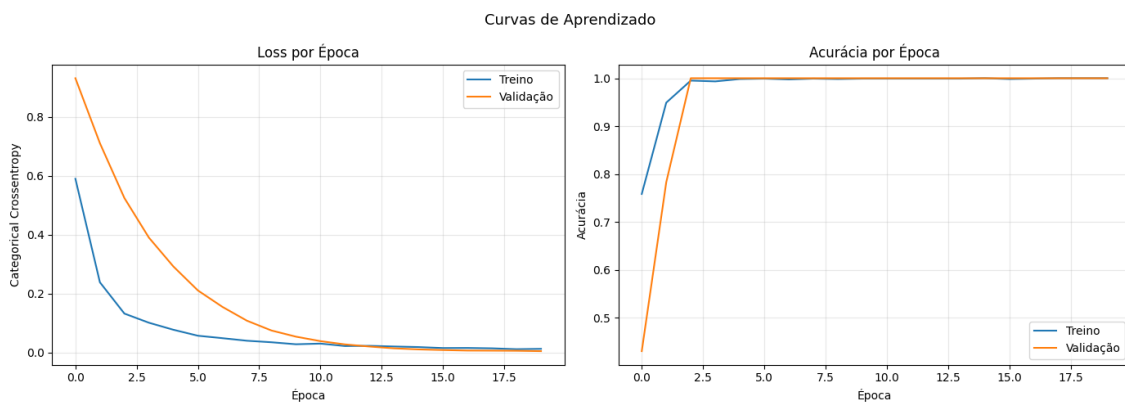


Figura 1. Curvas de perda e acurácia durante o treinamento.

As métricas utilizadas para avaliar o modelo no conjunto de teste foram acurácia, precisão, *recall* e F1-score, calculadas para cada classe individualmente e também como média ponderada. A matriz de confusão também foi gerada para complementar a análise dos resultados, apresentados na Seção 11.

11. Resultados Obtidos

11.1. Métricas no Conjunto de Teste

A avaliação final foi realizada sobre o conjunto de teste, composto por 258 janelas que o modelo nunca viu durante o treinamento, com 86 janelas de cada classe. Os resultados obtidos estão apresentados na Tabela 6.

Tabela 6. Métricas de desempenho no conjunto de teste

Classe	Precisão	Recall	F1-score	Suporte
Normal	1,00	1,00	1,00	86
Anômalo N1	1,00	1,00	1,00	86
Anômalo N2	1,00	1,00	1,00	86
Média ponderada	1,00	1,00	1,00	258

A acurácia final no conjunto de teste foi de 100%, com perda de 0,0066. Todas as classes obtiveram precisão, *recall* e F1-score iguais a 1, indicando que o modelo não cometeu nenhum erro de classificação sobre os 258 dados de teste.

11.2. Matriz de Confusão

A matriz de confusão, apresentada na Figura 2, confirma os resultados da tabela anterior, com as 86 amostras de cada classe classificadas corretamente, sem nenhuma confusão entre as classes *Normal*, *Anômalo N1* e *Anômalo N2*.

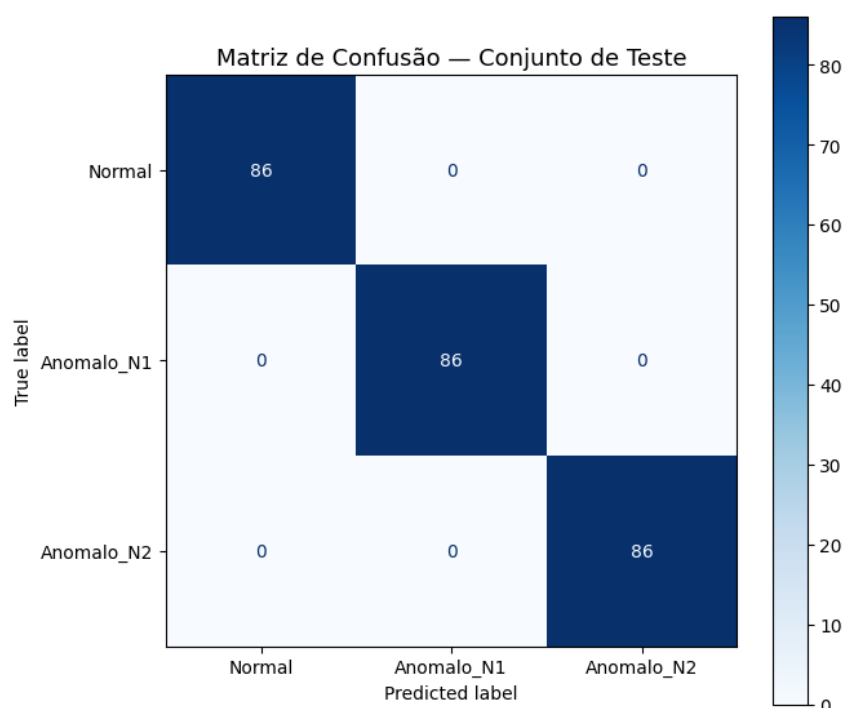


Figura 2. Matriz de confusão no conjunto de teste.

12. Análise dos Resultados

Os resultados obtidos foram expressivos, com acurácia de 100% no conjunto de teste e sem nenhum erro de classificação registrado na matriz de confusão. Ainda assim, alguns pontos merecem ser discutidos com mais cuidado antes de afirmar que o modelo está completamente resolvido.

O primeiro ponto é sobre a confiabilidade do resultado. Uma acurácia perfeita em problemas de classificação costuma levantar suspeitas de vazamento de dados ou overfitting. Neste trabalho, uma versão anterior do pipeline utilizava divisão aleatória dos

dados, o que permitia que janelas temporalmente adjacentes aparecessem simultaneamente no treino e no teste, inflando artificialmente os resultados. Após corrigir isso com a divisão temporal, o modelo manteve a acurácia de 100%, o que aumenta a confiança no resultado. Ainda assim, treino e teste partiram da mesma sessão de gravação, então não é possível garantir que o modelo generalize bem para gravações feitas em condições diferentes, como outro dia, outra fixação do sensor ou variações na rotação do motor.

Em relação ao overfitting, as curvas de treinamento não mostraram o padrão clássico de divergência entre perda de treino e de validação. O modelo convergiu de forma estável a partir da quinta época, sem sinais claros de memorização excessiva dos dados de treino. A oscilação observada nas primeiras épocas está associada à taxa de aprendizado inicial de 1×10^{-3} , que se mostrou um pouco alta para um modelo tão pequeno, causando passos grandes demais no início do treinamento. Isso não comprometeu o resultado final, mas poderia ser amenizado com uma taxa inicial menor ou com o uso de *learning rate warmup*.

Quanto ao dataset, o volume de dados gerado foi suficiente para o treinamento, resultando em 258 janelas bem distribuídas entre as três classes. O balanceamento natural das classes foi um ponto positivo, eliminando a necessidade de técnicas artificiais de balanceamento. Por outro lado, o fato de todos os dados virem de uma única sessão de coleta por classe limita a diversidade do conjunto e pode ser um fator relevante quando o sistema for testado em condições reais de operação.

As métricas escolhidas, acurácia, precisão, *recall* e F1-score, são adequadas para problemas de classificação multiclasse balanceada e cobrem bem os aspectos relevantes do desempenho do modelo. A matriz de confusão complementou a análise ao mostrar a distribuição dos erros por classe, mesmo que neste caso todos os valores tenham sido zero.

Entre as principais dificuldades encontradas na implementação, destaca-se a identificação do vazamento de dados causado pela divisão aleatória nas primeiras versões do código, que produzia resultados aparentemente perfeitos mas tecnicamente incorretos. Outro ponto foi o tratamento dos arquivos de entrada, que apresentavam número diferente de colunas entre si, exigindo uma leitura mais cuidadosa com seleção explícita das colunas de interesse.

Como melhorias futuras, seria interessante coletar dados em múltiplas sessões para aumentar a diversidade do dataset e testar a generalização do modelo de forma mais rigorosa. Avaliar o desempenho com diferentes tamanhos de janela e explorar representações no domínio da frequência via FFT também são caminhos que podem trazer ganhos, especialmente considerando que as distribuições de amplitude dos sinais brutos são bastante similares entre as classes, como observado na análise exploratória.

13. Implementação em TinyML

Esta seção descreve a etapa de conversão, quantização e embarque do modelo treinado no microcontrolador Raspberry Pi Pico W, completando o pipeline que havia sido apenas planejado anteriormente. Diferente da etapa de treinamento, realizada em Python no Google Colab, a implementação embarcada foi desenvolvida em C/C++ utilizando o Pico SDK e o framework TensorFlow Lite Micro, em um projeto criado no Visual Studio Code com a extensão oficial do Raspberry Pi Pico.

13.1. Quantização e Exportação do Modelo

O modelo treinado em ponto flutuante de 32 bits, com 6.467 parâmetros e tamanho de aproximadamente 25 KB, foi convertido para o formato TensorFlow Lite por meio do *TFLiteConverter*. Inicialmente, gerou-se uma versão quantizada do modelo, com tamanho de 15,58 KB. Essa quantização foi adotada por dois motivos principais: reduzir o consumo de memória flash do microcontrolador e acelerar a inferência, já que o RP2040, processador presente no Raspberry Pi Pico W, não possui unidade de ponto flutuante dedicada.

O modelo quantizado foi então convertido para um vetor de bytes em linguagem C, por meio do utilitário *xxd*, gerando o arquivo *drone_fault_model.cc*, com 97,19 KB. Esse tamanho maior em relação ao arquivo *.tflite* original é esperado, já que o arquivo *.cc* representa os bytes do modelo em formato de texto, como uma sequência de valores hexadecimais, e não como dados binários compactos.

13.2. Arquitetura do Firmware Embarcado

O firmware desenvolvido para o Raspberry Pi Pico W é responsável por três tarefas principais: adquirir os sinais de vibração do sensor MPU-6050, normalizar e organizar esses sinais em janelas compatíveis com a entrada do modelo, e executar a inferência localmente por meio do TensorFlow Lite Micro.

A comunicação com o MPU-6050 é feita via barramento I2C, configurado a 400 kHz. Na inicialização, o sensor é retirado do modo de repouso e configurado para operar com escala de $\pm 2g$ no acelerômetro. A leitura dos três eixos de aceleração é realizada por meio de uma transação I2C que lê seis bytes consecutivos do sensor, convertidos posteriormente para unidades de m/s^2 . A taxa de amostragem é controlada por um atraso fixo de 10 ms entre leituras consecutivas, resultando em uma frequência aproximada de 100 Hz, a mesma utilizada durante a coleta do dataset de treinamento. Antes de iniciar o laço principal, o firmware realiza uma medição da taxa real de amostragem, contabilizando o tempo decorrido para a aquisição de uma janela completa, o que permite verificar em tempo de execução se a frequência efetiva está próxima da esperada.

Cada janela de 26 amostras coletadas é normalizada utilizando os mesmos parâmetros de média e desvio padrão obtidos pelo *StandardScaler* durante o treinamento, conforme discutido na Seção 9. Esses parâmetros foram extraídos do ambiente de treinamento e embarcados como constantes fixas no firmware, um valor de média e um de desvio padrão por eixo, evitando a necessidade de recalcular estatísticas de normalização no microcontrolador.

Para executar a inferência, o firmware instancia um interpretador do TensorFlow Lite Micro, alocando uma área de memória de 60 KB para os tensores intermediários do modelo. As operações necessárias para a execução do modelo foram registradas manualmente por meio de um *MicroMutableOpResolver*, totalizando dez operações: convolução bidimensional, convolução *depthwise*, *reshape*, *softmax*, *mean*, *max pooling* bidimensional, camada densa, quantização, expansão de dimensões e desquantização. O registro explícito apenas das operações utilizadas pelo modelo, em vez de um resolvidor genérico com todas as operações disponíveis, contribui para reduzir o consumo de memória do firmware.

A cada nova janela coletada e normalizada, os dados são copiados para o tensor de entrada do interpretador, a inferência é executada, e a classe prevista é determinada a partir do índice de maior probabilidade entre as três saídas do modelo, correspondentes às classes *Normal*, *Anômalo N1* e *Anômalo N2*. O resultado é então exibido pela saída serial, junto com as probabilidades associadas a cada classe.

13.3. Compilação e Tamanho do Firmware

O firmware foi compilado com sucesso utilizando o Pico SDK, gerando os arquivos de saída padrão da plataforma, incluindo o arquivo *drone_fault_detector.uf2*, utilizado para gravar o firmware no microcontrolador. O tamanho final desse arquivo foi de 479,7 KB, valor compatível com os 2 MB de memória flash disponíveis no Raspberry Pi Pico W, restando margem suficiente para eventuais ajustes futuros no firmware ou no modelo embarcado.

13.4. Validação Prática no Protótipo

Após a compilação e a gravação do firmware no Raspberry Pi Pico W, o sistema foi testado de forma prática no protótipo montado em bancada, composto pelo microcontrolador, o sensor MPU-6050 fixado próximo ao motor e a estrutura de suporte utilizada durante a coleta dos dados. A Figura 3 apresenta o protótipo completo utilizado nos testes.

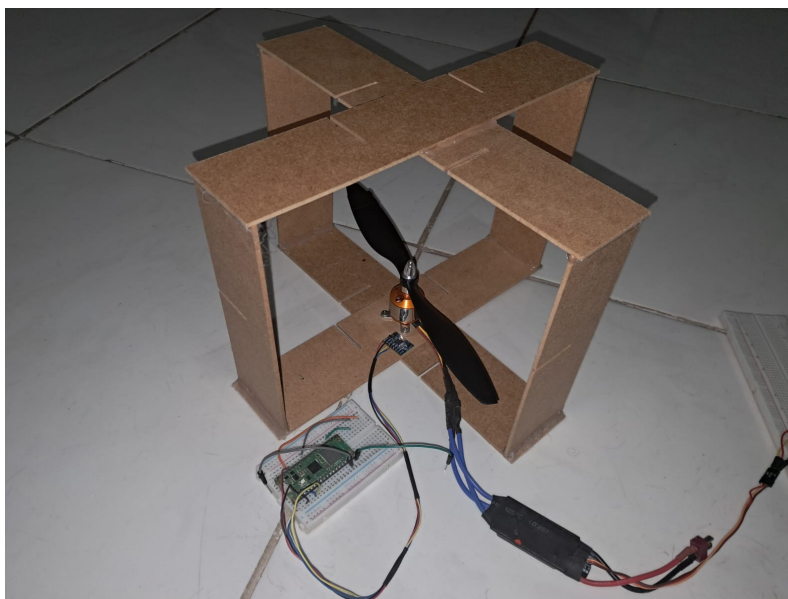


Figura 3. Protótipo montado em bancada, com o Raspberry Pi Pico W, o sensor MPU-6050 e o motor de drone utilizados nos testes.

Com o motor em funcionamento, o firmware foi observado em execução por meio do monitor serial, exibindo em tempo real a classe prevista para cada janela de 26 amostras coletada, juntamente com as probabilidades associadas a cada uma das três classes. A Figura 4 apresenta um trecho da saída do monitor serial durante um dos testes realizados, evidenciando o funcionamento contínuo do sistema de classificação diretamente no microcontrolador, sem qualquer dependência de processamento externo.


```

=== Drone Fault Detector ===
MPU6050 inicializado
Modelo carregado. Iniciando coleta...

fs real: 97.7 Hz
Resultado: Normal      (N:1.00  N1:0.00  N2:0.00)
Resultado: Normal      (N:1.00  N1:0.00  N2:0.00)
Resultado: Normal      (N:1.00  N1:0.00  N2:0.00)
Resultado: Normal      (N:1.00  N1:0.00  N2:0.00)
Resultado: Normal      (N:0.97  N1:0.02  N2:0.01)
Resultado: Anomalo_N2  (N:0.02  N1:0.02  N2:0.96)
Resultado: Anomalo_N2  (N:0.00  N1:0.00  N2:1.00)
Resultado: Anomalo_N2  (N:0.00  N1:0.00  N2:1.00)
Resultado: Anomalo_N2  (N:0.01  N1:0.05  N2:0.94)
Resultado: Anomalo_N1  (N:0.14  N1:0.46  N2:0.40)

```

Figura 4. Saída do monitor serial mostrando a classificação em tempo real realizada pelo firmware embarcado.

Esses testes confirmaram que o pipeline completo, desde a leitura do acelerômetro até a inferência com o TensorFlow Lite Micro, funciona de maneira integrada no dispositivo embarcado, validando na prática a viabilidade da solução proposta para a detecção de falhas em motores de drone com TinyML.

13.5. Latência e Considerações sobre Tempo Real

A janela de entrada do modelo corresponde a 26 amostras coletadas a aproximadamente 100 Hz, o que representa cerca de 260 ms de sinal necessários antes de cada inferência. O tempo de execução da própria inferência, ou seja, o tempo gasto pela chamada *Invoke()* do interpretador, ainda não foi instrumentado e medido diretamente no firmware atual. Com base em valores reportados na literatura para modelos de complexidade semelhante executados em microcontroladores equivalentes [Arciniegas et al. 2025], espera-se que esse tempo seja da ordem de poucas dezenas de milissegundos, o que resultaria em uma latência total do sistema, da coleta da janela até a obtenção do resultado da classificação, inferior a 300 ms. A medição direta desse tempo no firmware é apontada como um passo pendente, discutido com mais detalhe na Seção 12.

14. Reprodutibilidade

A reprodução completa deste trabalho envolve duas etapas distintas: o treinamento do modelo em ambiente Python, realizado no Google Colab, e a implementação do firmware embarcado, desenvolvida em C/C++ com o Pico SDK. O código de ambas as etapas está disponível publicamente.

14.1. Treinamento do Modelo

O notebook com a implementação completa do treinamento está disponível no Google Colab, acessível pelo link: https://colab.research.google.com/drive/1IZiZG0CSGYFVTtZQ3nSK9eL3_bvZ0bjL?usp=sharing.

14.2. Firmware Embarcado

O código-fonte do firmware desenvolvido para o Raspberry Pi Pico W, incluindo a aquisição de dados via MPU-6050, a normalização das janelas e a execução da inferência com o TensorFlow Lite Micro, está disponível no seguinte repositório do GitHub: <https://github.com/AndreAlves-18/tinymml-drone-vibration-detector.git>.

14.3. Ambiente de Execução

O código de treinamento foi desenvolvido e executado no Google Colab, utilizando Python 3.12. As principais bibliotecas e suas versões estão listadas na Tabela 7.

Tabela 7. Bibliotecas utilizadas e suas versões

Biblioteca	Versão
TensorFlow	2.20.0
Keras	3.13.2
NumPy	2.0.2
Pandas	2.2.2
Scikit-learn	1.6.1
Matplotlib	3.10.0
Seaborn	0.13.2
OpenPyXL	3.1.5

O firmware embarcado foi desenvolvido em ambiente Linux Ubuntu/Debian, utilizando o Visual Studio Code com a extensão oficial do Raspberry Pi Pico, o Pico SDK e o framework TensorFlow Lite Micro, por meio do repositório *pico-tflmicro*.

14.4. Instruções para Execução

Para reproduzir os resultados do treinamento, basta acessar o link do notebook e seguir os passos abaixo:

1. Fazer o upload dos arquivos de dados coletados pelo MPU-6050 para o ambiente do Colab, ou montar o Google Drive com os arquivos na pasta correta;
2. Atualizar as variáveis de caminho dos arquivos na seção de configuração do notebook;
3. Executar as células na ordem, de cima para baixo, respeitando a sequência das seções.

Para reproduzir o firmware embarcado, é necessário clonar o repositório do GitHub indicado acima, configurar o Pico SDK conforme as instruções do próprio repositório, compilar o projeto com o CMake e o Ninja, e gravar o arquivo *.uf2* resultante no Raspberry Pi Pico W em modo *bootloader*.

14.5. Organização do Notebook

O notebook está dividido em seções numeradas que seguem o mesmo fluxo descrito neste trabalho, conforme apresentado na Tabela 8.

Tabela 8. Organização do notebook

Seção	Conteúdo
1	Instalação e importação de bibliotecas
2	Variáveis de configuração e hiperparâmetros
3	Carregamento dos arquivos de dados
4	Pré-processamento: janelamento e normalização
5	Divisão temporal treino, validação e teste
6	Definição da arquitetura do modelo
7	Treinamento e curvas de aprendizado
8	Avaliação e métricas no conjunto de teste
9	Quantização e exportação para TensorFlow Lite
10	Geração do arquivo C para embarque

15. Conclusão

Este trabalho apresentou a implementação completa de um sistema de detecção de falhas em hélices de drone com TinyML, partindo da coleta de sinais de vibração com o sensor MPU-6050 até a execução da inferência diretamente no microcontrolador Raspberry Pi Pico W. Diferente da etapa anterior, em que a implantação embarcada ainda era um trabalho planejado, neste entregável o pipeline foi efetivamente concluído de ponta a ponta, abrangendo coleta, pré-processamento, treinamento, quantização e embarque do modelo.

O modelo utilizado foi a mesma CNN 1D compacta definida anteriormente, adaptada para operar sobre janelas de 26 amostras dos eixos A_x , A_y e A_z , coletadas a aproximadamente 100 Hz, taxa imposta pela limitação de transmissão serial do protótipo. Mesmo com essa adaptação, o modelo manteve os mesmos 6.467 parâmetros treináveis, demonstrando que a arquitetura escolhida se mostrou robusta a mudanças na resolução temporal da entrada.

Os resultados obtidos com o dataset próprio, coletado em bancada nas três condições de operação consideradas, foram satisfatórios, com acurácia de 100% no conjunto de teste, precisão, *recall* e F1-score iguais a 1,00 para todas as classes, e perda de 0,0066. Após a quantização, o modelo ocupou 16,88 KB em formato inteiro de 8 bits, e o firmware completo, incluindo o TensorFlow Lite Micro e toda a aplicação embarcada, resultou em um arquivo de 479,7 KB, valor compatível com a memória flash de 2 MB disponível no Raspberry Pi Pico W.

Entre as limitações identificadas, destacam-se três pontos principais. O primeiro é o volume reduzido do dataset próprio em comparação ao dataset de referência utilizado anteriormente, consequência direta da taxa de amostragem mais baixa imposta pela comunicação serial. O segundo é o vazamento de dados identificado no procedimento de normalização, em que o *StandardScaler* foi ajustado sobre o conjunto completo de janelas antes da divisão entre treino, validação e teste, o que pode ter contribuído para a acurácia perfeita observada. O terceiro é a ausência de medição direta do tempo de inferência no microcontrolador, restando apenas estimativas baseadas em valores reportados na literatura para dispositivos equivalentes.

Como melhorias planejadas, pretende-se corrigir o procedimento de normalização, ajustando o *scaler* exclusivamente sobre o conjunto de treino antes

de aplicá-lo aos conjuntos de validação e teste, de modo a eliminar o vazamento identificado. Pretende-se também instrumentar o firmware para medir diretamente o tempo de execução da inferência, permitindo calcular com precisão a latência total do sistema. Por fim, considera-se relevante coletar dados em múltiplas sessões e com controle sobre a rotação do motor, de forma a avaliar de maneira mais rigorosa a capacidade de generalização do modelo em condições mais próximas do uso prático em drones.

Apesar dessas limitações, o trabalho demonstrou que é possível executar um modelo de aprendizado de máquina diretamente em um microcontrolador de baixo custo, sem dependência de processamento externo, validando a viabilidade da abordagem de TinyML para a detecção de falhas em motores de drone a partir de sinais de vibração.

Declaração de Uso de Inteligência Artificial

Durante a elaboração deste trabalho, foi utilizado o assistente de IA Claude (Anthropic) como ferramenta de apoio à escrita. O uso se restringiu a tarefas como identificação de erros gramaticais, revisão de coesão textual e sugestões de reformulação de trechos.

Referências

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software disponível em [tensorflow.org](https://www.tensorflow.org).
- Adaika, Z., Al-Haddad, L. A., Giernacki, W., Jaber, A. A., Boumehraz, M., Hamzah, M. N., and Flayyih, M. A. (2025). Fault detection and diagnosis methodologies for unmanned aerial vehicles: State-of-the-art. *Journal of Intelligent & Robotic Systems*, 111(2):63.
- Al-Haddad, L. A. (2025). Dataset on UAV fault diagnosis using vibrational signals.
- Al-Haddad, L. A., Jaber, A. A., Mahdi, N. M., Al-Haddad, S. A., Al-Karkhi, M. I., Al-Sharify, Z. T., and Ogaili, A. A. F. (2024). Protocol for uav fault diagnosis using signal processing and machine learning. *STAR Protocols*, 5(4).
- Arciniegas, S., Rivero, D., Piñan, J., Diaz, E., and Rivas, F. (2025). Iot device for detecting abnormal vibrations in motors using tinyml. *Discover Internet of Things*, 5(1):41.
- Ceylan, N., Sönmez, E., and Kaçar, S. (2024). Cost effective detection of uneven mounting fault in rotary wing drone motors with a cnn based method. *Signal, Image and Video Processing*, 18(11):8049–8059.
- David, R., Duke, J., Jain, A., Janapa Reddi, V., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Wang, T., Warden, P., and Rhodes, R. (2021). Tensorflow lite micro: Embedded machine learning for tinyml systems. In Smola, A., Dimakis, A., and Stoica, I., editors, *Proceedings of Machine Learning and Systems*, volume 3, pages 800–811.
- Fawaz, H. I. (2020). Timeseries classification from scratch. https://keras.io/examples/timeseries/timeseries_classification_from_scratch/. Keras Examples. Acesso em: maio 2026.

- Lin, J., Zhu, L., Chen, W.-M., Wang, W.-C., and Han, S. (2023). Tiny machine learning: Progress and futures [feature]. *IEEE Circuits and Systems Magazine*, 23(3):8–34.
- Ozkat, E. C. (2024). Vibration data-driven anomaly detection in uavs: A deep learning approach. *Engineering Science and Technology, an International Journal*, 54:101702.
- Puchalski, R. and Giernacki, W. (2022). Uav fault detection methods, state-of-the-art. *Drones*, 6(11).
- RAJAKANNU, A., KAMARUDDEN, M., and KP, R. (2024). Intelligent fault diagnosis of rotating machinery using deep learning algorithms: A comparative analysis of mlp, cnn, rnn, and lstm. *INTERNATIONAL JOURNAL*, 11(9):294–315.
- Tiboni, M., Remino, C., Bussola, R., and Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3).